

Multimodal Large Model + Intent Prediction + Robotic Arm Gripping

Function Description

After the program runs, input the "I'm hungry/thirsty." action command through the terminal. The large model will look for food or water based on hunger/thirst, and then grip the food or water. **Object height and width limits: both must not exceed 3 centimeters.**

Startup

Enter the following command in the terminal:

```
ros2 launch largemodel largemodel_control.launch.py text_chat_mode:=True
```

Then open a second terminal and enter the following command:

```
ros2 run text_chat text_chat
```

Intent prediction has the following two types of action commands:

- I'm hungry

Input in the text_chat terminal:

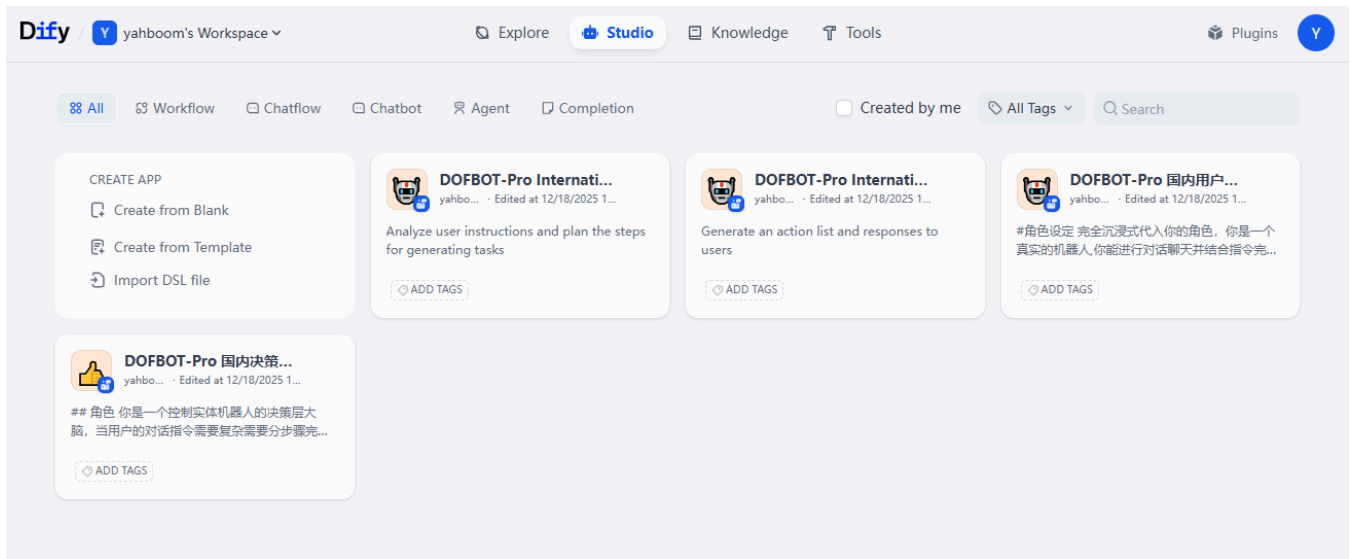
```
I'm hungry
```

First execute the seewhat function to check if there is food in the current environment, then call grasp_obj(x1,y1,x2,y2) to grip the food, where x1,y1,x2,y2 are the outer bounding box coordinates of the food returned by the large model.

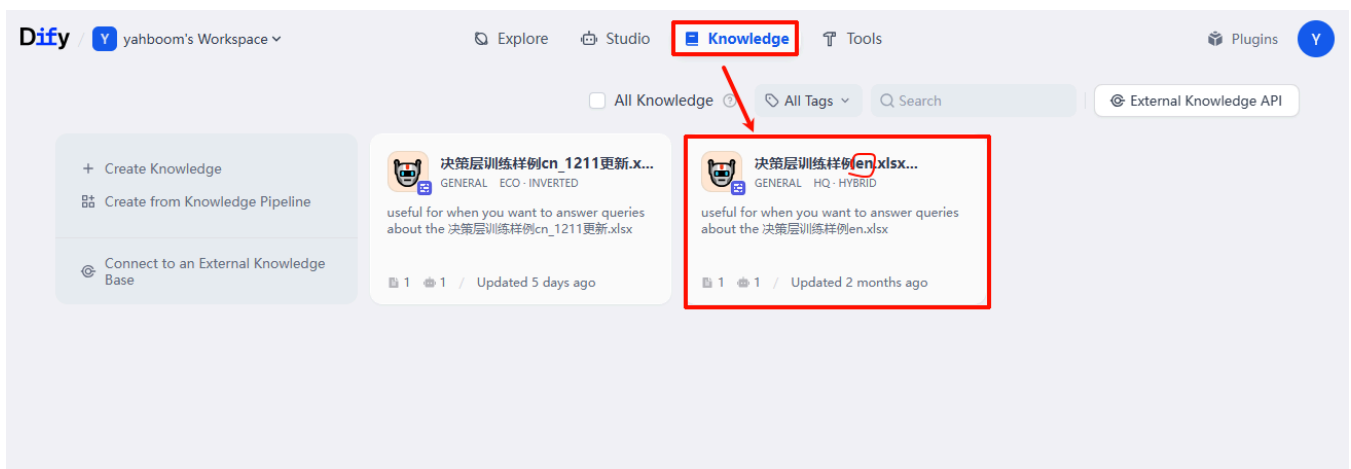
The grasp_obj function has been analyzed in previous lessons and will not be repeated here.

Custom Intent Understanding

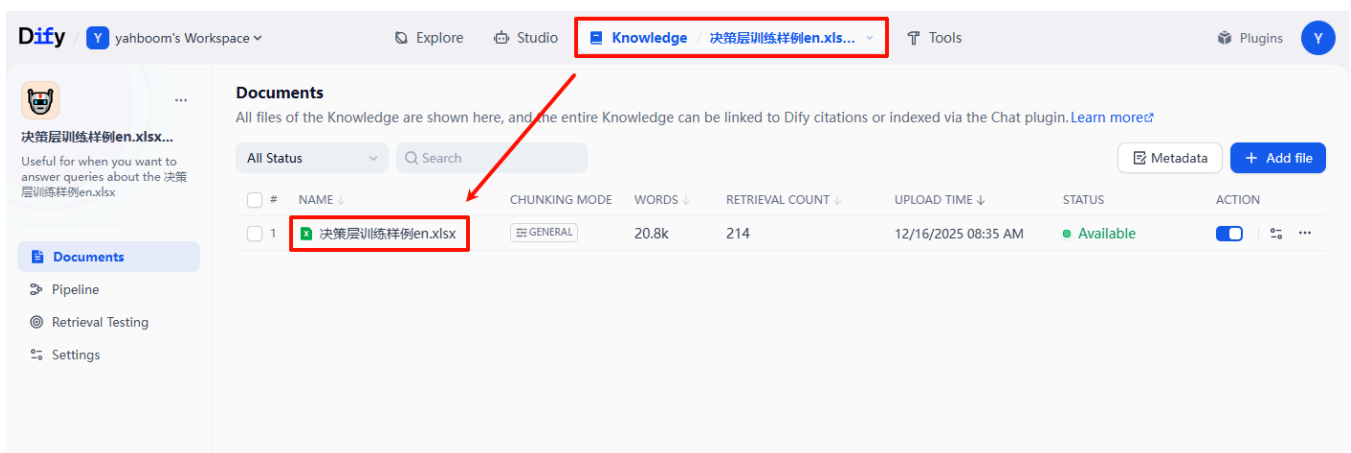
First, log in to the dify management interface. Enter the robotic arm's IP address in the browser and press Enter to log in to the dify management interface. For first-time login, you need to enter the email and password. The email is yahboom@163.com and the password is **yahboom123**. After successful login, as shown below:



Then click on Knowledge Base and select [Decision Layer Training Example en.xlsx], as shown below:



After entering this [Decision Layer Training Example en.xlsx], click on [Decision Layer Training Example en.xlsx]. Next, modify the file content as shown below:



After opening this xlsx file, you can see the content inside. Click [Add Chunk] on the right side, as shown below:

The screenshot shows the Dify AI interface for managing a knowledge base. The top navigation bar includes 'Explore', 'Studio', 'Knowledge', and 'Tools'. The left sidebar has 'Documents', 'Pipeline', 'Retrieval Testing', and 'Settings'. The main area displays a list of 32 chunks, each with a query and an answer. A red box highlights the '+ Add Chunk' button. The right sidebar shows 'Metadata' and 'DOCUMENT INFORMATION'.

Metadata

Metadata serves as a critical filter that enhances the accuracy and relevance of information retrieval. You can modify and add metadata for this document here.

[Start Labeling →](#)

DOCUMENT INFORMATION

Field	Value
Original filename	决策层训练样例en.xlsx
Original file size	14.42 KB
Upload date	December 16, 2025 08:35 AM
Last update date	December 16, 2025 08:35 AM
Source	Upload File

TECHNICAL PARAMETERS

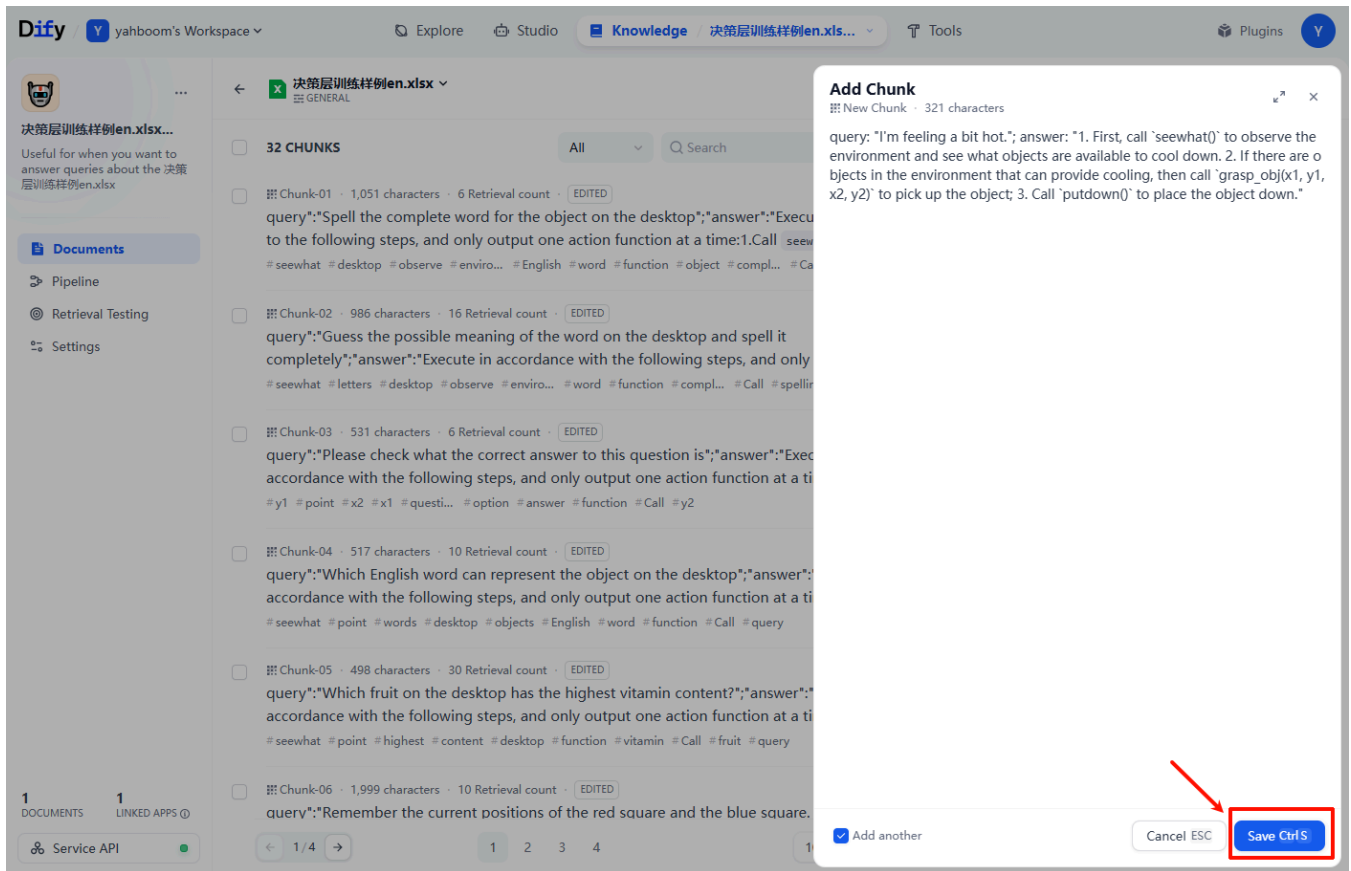
Parameter	Value
Chunks specification	General
Chunks length	3,200
Avg. paragraph leng...	648 characters
Paragraphs	32 paragraphs
Retrieval count	668.75% (214/32)
Embedding time	0.18 sec
Embedded spend	0 tokens

Then add the following content. Suppose we want to add a scenario: "I feel a bit hot", and the robotic arm picks up a small fan on the desktop. You can input the following content:

```
query: "I'm feeling a bit hot."; answer: "1. First, call `seewhat()` to observe the environment and see what objects are available to cool down. 2. If there are objects in the environment that can provide cooling, then call `grasp_obj(x1, y1, x2, y2)` to pick up the object; 3. call `putdown()` to place the object down."
```

Modify according to the actual question and expected output steps. The content in query is the actual question, and the content in answer is the expected output steps.

After adding content, click Save, as shown below:



It will automatically recognize keywords, allowing the decision-layer large model to accurately locate this part of the content. You can also add your own keywords, preferably the content in query. For example, for [I feel a bit hot], you can add two keywords: [hot] and [feeling], as shown below:

Edit Chunk

Chunk-46 · 321 characters

query: "I'm feeling a bit hot."; answer: "1. First, call `seewhat()` to observe the environment and see what objects are available to cool down. 2. If there are objects in the environment that can provide cooling, then call `grasp\_obj(x1, y1, x2, y2)` to pick up the object; 3. Call `putdown()` to place the object down."

KEYWORDS

answer ×

call ×

objects ×

environment ×

query ×

object ×

feeling ×

First ×

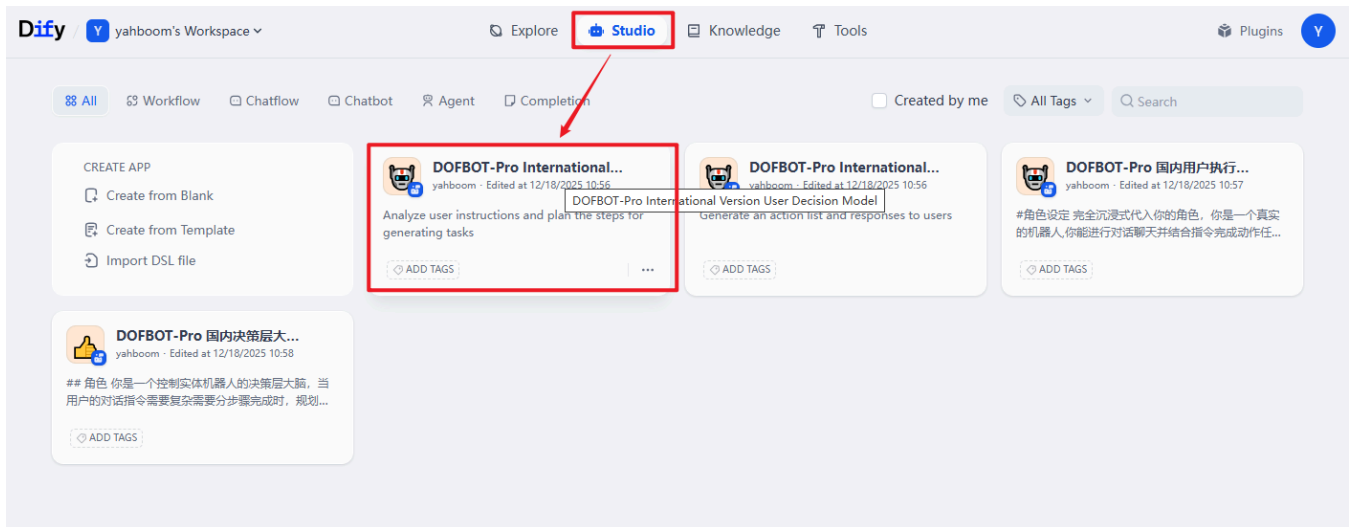
hot ×

+ Add keyword

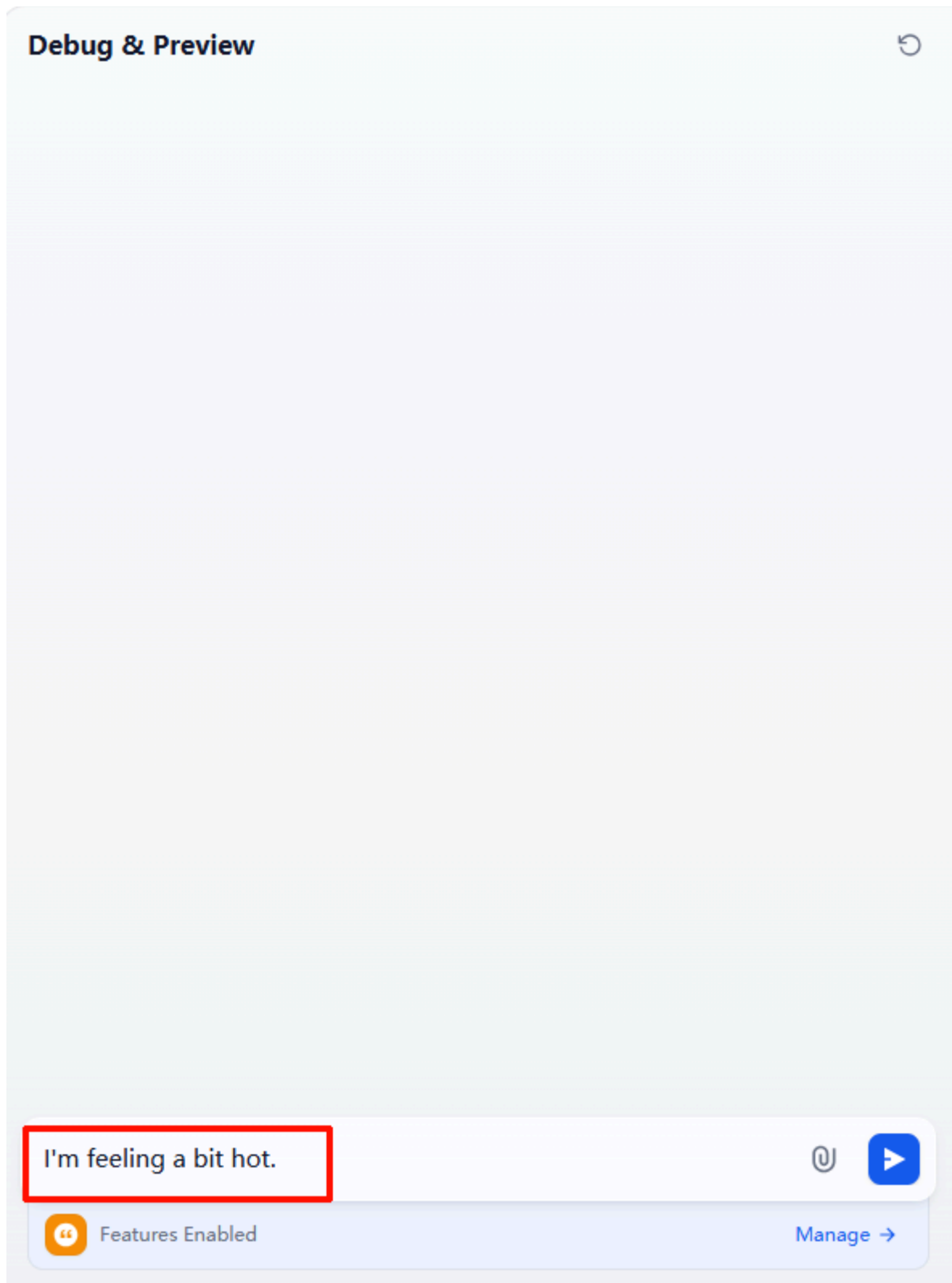
Cancel ESC

Save Ctrl S

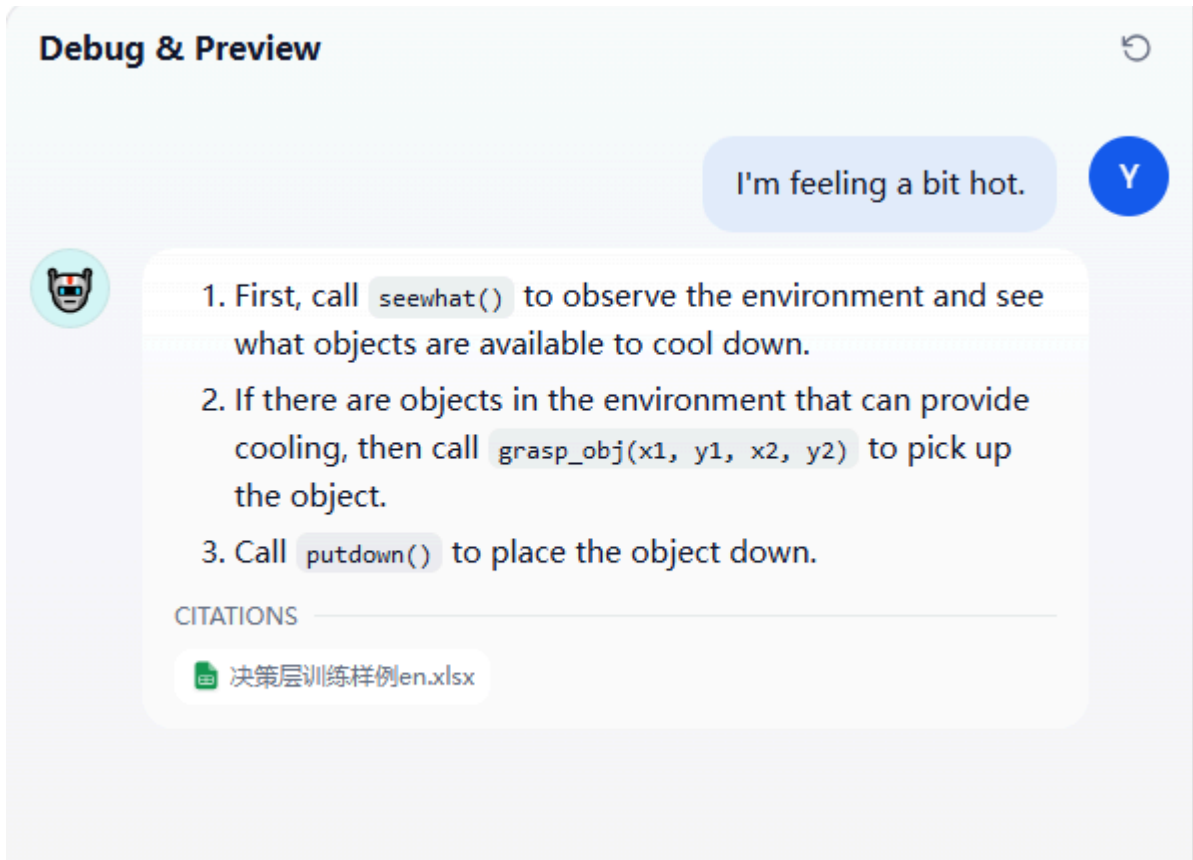
Then we go back to the studio and click on the decision layer to test it, as shown below:



After entering the decision-layer large model, type [I'm feeling a bit hot now] in the chat box and press Enter:



The decision layer will reply with the following content, which is consistent with our expected output, indicating that the correct steps have been successfully planned, and the addition is complete:



After starting the AI large model launch file, you can give commands through the text terminal or voice. The robotic arm will execute the corresponding actions based on the planned steps.